

The State of Agentic Engineering

A short report on the state of agentic engineering adoption in more advanced tech orgs, June 2026, Sebastian Heide-Meyer zu Erpen

How to use this document

Do not read the full document - unless you actually want to. Rather read the summary and then read the specific chapter(s) you are interested in to get more details. This report is based on a lot of self reporting by companies. If you want to look into company specifics, use the talk-/podcast summaries in the [Annex](#) and/or go to the real thing, based on the links provided in the Annex company sections or the [list of references](#). Note: the Upvest and Mapbox podcast episodes are in German.

Summary

This report summarises several talks, panel discussions, podcast episodes and articles that contain relevant information on how bigger tech organisations are adopting agentic engineering, looking at (Success Measurements, Harnesses, Tool use, Governance, Security and Compliance). All references are included. Summaries of the most important sources are added to the Annex of this document. **All sources are from Feb-May 2026.**

Evidence for the success of agentic engineering is still mostly directional rather than robustly causal.¹ The public talks show meaningful signals - higher throughput, faster onboarding - but these signals remain sensitive to team context, tech excellence and organisational maturity.

Frameworks that have been used to [measure the productivity](#) of digital product organizations before agentic, largely still hold true. **DORA** is one of the most widely adopted and with relevant adjustments is still a very potent framework for measuring productivity of software engineering teams. In general a stable **scorecard** across four dimensions: **outcome** (OKR delivery, business impact), **output** (delivery throughput), **tech health** (stability, quality, cost) and **team health** (developer happiness and confidence) is suggested.

The leading indicator for positive impact appears to be technical excellence.¹ Organisations with strong tests, CI/CD, observability, ownership, review discipline and fast feedback loops seem to see a general uplift in output metrics². A core tenet is: the engineer is responsible for the PR - independent from the tools used to generate the PR. Near-term use

¹ Laura Tacho (DX), The Pragmatic Summit <https://www.youtube.com/watch?v=LOHgRw43fFk>

² Niklas Gustavsson (Spotify), Code with Claude <https://youtu.be/zFslvuvYifQ?t=170>

cases are often focusing specifically on toil tasks³ - migrations, repetitive refactorings, bug fixes, test generation - or internal tooling⁶ where success can be verified deterministically.

Tool choice remains open in early adoption, but **mature organisations seem to converge on a small number of blessed harnesses** and workflows.^{4 5} The general approach still focuses on openness in order to not impede adoption. Skills, Workflows, Subagents and other tooling are being shared internally via simple shared repositories, tools like skillshare⁶ or via platform approaches in bigger companies.

Security and Compliance are treated with more rigor in bigger and more regulated⁷ companies. In these companies **MCP Gateways** or **AI Gateways**⁸ are being utilized to enforce guidelines. The general notion is: the agent can only have the same access rights as the human using it. Rigorous RBAC and Zero Trust, auditability, and gateway-style controls are therefore foundational for safer agentic engineering.

Costs are only starting to become a topic but there is no clear insight on how companies are reigning in costs yet. Model governance (bigger, more expensive for high complexity tasks) and running open weights models internally, have not been mentioned by any of the companies yet.

Introduction

This report outlines the findings from researching the state of the art on agentic engineering from public sources. Scientific research on the topic is exploding but the insight from existing studies is insignificant since studies are backward looking and all (pre-)published studies predate the rise of Claude Code and similar coding harnesses and the newest model generations which changed agentic software engineering significantly.⁹ Hence this report focuses primarily on practitioner evidence, public company writeups, and observable operating-model patterns rather than pretending that the academic evidence has already stabilized.

Agentic engineering is still in its early innings. Most companies already report high adoption but low transformation.¹⁰ Even the more advanced companies are still in a somewhat probe-and-response phase. While there is no clear framework yet for utilizing agentic engineering in bigger software orgs, it is clear that tech orgs and the collaboration within tech

³ Ty Smith & Anshu Chada (Uber), The Pragmatic Summit <https://youtu.be/i1tZN41VKcE?t=411>

⁴ Harness is the control pane that manages the context, interactions, tools and security restrictions around the LLM as part of an agent (agent = harness + LLM)

⁵ Annex 3: [Mapbox Case Summary](#)

⁶ <https://github.com/runkids/skillshare>

⁷ Annex 2: [Upvest Case Summary](#)

⁸ <https://blog.cloudflare.com/internal-ai-engineering-stack/>

⁹ <https://ingoeichhorst.medium.com/stand-der-ki-coding-effizienz-2026-a248587ef94a>

¹⁰ Laura Tacho (DX), The Pragmatic Summit <https://youtu.be/LOHgRw43fFk?t=651>

orgs and with stakeholders is changing. Between the inner loop (coding) and the outer loop (delivery), a new middle loop is forming as Thoughtworks describes it¹¹.

The role of the engineer is shifting. The center of gravity is moving away from solving low-level isolated technical problems toward understanding user needs, business requirements, system boundaries and the existing architecture - and then turning that understanding into sustainably maintainable, evolvable software that meets the users' needs and the business's constraints.

Put differently: the future engineer will spend less time on “how do I center a div?” and more time on “what exactly are we trying to achieve, what are the constraints, what is the right system change, and how do we verify that it is safe and correct?”

AI amplifies the underlying engineering system. Laura Tacho (DX) says at the Pragmatic Summit: “AI is an accelerator”¹² and reports that while companies are reporting 2x the amount of incidents, other companies report 50% fewer incidents. If a team is already technically excellent, AI tends to make it faster and even has a positive impact on quality. If a team already struggles with tech debt, weak review discipline, poor tests, or unclear ownership, AI often accelerates that dysfunction as well.

Anthropic's own internal framing reinforces the same point: once code generation becomes cheap, the binding constraints move toward verification, review, security, ownership, coordination, and product judgment rather than raw implementation bandwidth.¹³

Agentic engineering should not be seen primarily as a tooling topic, it is an operating-model topic.

Success measurement

There is currently no strong industry-wide causal evidence for the “right” measurement model. Most organisations are still early, and many reported gains are anecdotal. Laura Tacho's Pragmatic Summit talk adds a useful counterweight here: adoption can be very high while measured gains remain modest, unevenly distributed, and highly sensitive to team context, codebase quality, and the chosen metric.

She suggests specific metrics for agentic engineering around Utilization, Impact and Cost:

¹¹ Thoughtworks: [The Future of software development retreat - key takeaways](#)

¹² Laura Tacho (DX), The Pragmatic Summit <https://youtu.be/LOHgRw43fFk?t=540>

¹³ Fiona Fong (Anthropic), Code with Claude <https://www.youtube.com/watch?v=IA5LWIGqnyM>

Utilization How much are developers adopting and utilizing AI tools?	Impact How is AI impacting engineering productivity?	Cost Is our AI spend and return on investment optimal?
- AI tool usage (DAUs/WAUs) - Percentage of PRs that are AI-assisted - Percentage of committed code that is AI-generated - Tasks assigned to agents *	- AI-driven time savings (dev hours/week) - Developer satisfaction - DX Core 4 metrics, including: - PR throughput - Perceived rate of delivery - Developer Experience Index (DXI) - Code maintainability - Change confidence - Change fail percentage - Human-equivalent hours (HEH) of work completed by agents *	- AI spend (both total and per developer) - Net time gain per developer (time savings - AI spend) - Agent hourly rate (HEH / AI spend) *

All observed companies report or imply output improvements

The companies consistently report increased throughput, productivity or automation capacity.¹⁴ This appears in different forms: higher PR or merge-request activity, more frequent AI-tool usage, stronger developer productivity sentiment, automated maintenance work at very large scale, faster internal tooling, and increased code or workflow output. Spotify and Cloudflare provide the strongest quantitative evidence, with Spotify reporting very high weekly AI-tool usage, strong productivity sentiment, substantially increased PR frequency, and millions of automated maintenance PRs through FleetShift, while Cloudflare reports broad R&D adoption, millions of AI requests, thousands of active internal users, and a material rise in weekly merge-request volume.

Most companies agree that output volume is not the final metric

The shared caveat is just as important as the productivity signal: more generated code, more PRs, or more automated work does not automatically mean better engineering outcomes. Review, verification, ownership, quality, and business impact become more important as AI increases output. Anthropic states this most generally by framing agentic coding as an operating-model challenge: planning, ownership, review, and verification norms have to change when code becomes easier to generate. Mapbox, Upvest, Uber, Spotify, and Anthropic all point in similar directions, even when they use different language.

Only Uber publicly highlights unresolved ROI at this level of specificity

Several companies imply that outcome measurement is difficult, but Uber is the only one that publicly frames the ROI problem with this level of directness.¹⁵ Uber reports strong adoption and productivity signals while also saying that AI costs have risen sharply since 2024 and that cost justification has moved beyond normal engineering budgets. The case therefore highlights the

¹⁴ See Annex 1. [Agentic Engineering Case Studies - Comparative Summary Report](#)

¹⁵ <https://fortune.com/2026/05/26/uber-coo-ai-spending-tokens-claude-code/>

next measurement frontier for agentic engineering: connecting usage, generated output, developer sentiment, and velocity to durable business outcomes.

Suggestion

Overall suggested KPIs from before the rise of agentic engineering remain relevant with the addition of AI-related metrics. A practical scorecard should remain stable across four dimensions:

1. **Outcome:** customer value, revenue impact, time to ship meaningful product changes, reliability of critical journeys, strategic speed.
2. **Output:** PRs, tickets completed, cycle time, deployment throughput, delivery volume.
3. **Tech health:** uptime, incident rate, rollback rate, change-failure rate, static-analysis signals, test coverage, mean time to verification.
4. **Team health:** developer satisfaction, perceived effectiveness, cognitive load, interruption burden, ownership, and confidence in generated code.

A pragmatic approach is to combine throughput metrics with lightweight recurring developer-experience surveys. Anthropic's Fiona Fung adds an adjacent warning: old processes may not fail loudly, but can quietly stop serving their purpose once AI changes the economics of prototyping, review, and implementation.

How the industry is adopting agentic engineering

The clearest pattern from the cases reviewed is that the organisations are moving toward governed workflows, explicit rules and distinctive tooling while trying to maintain developer autonomy. This is visible in different forms at Upvest, Mapbox, Cloudflare, Atlassian's framing of an AI-native SDLC with internal agents across coding, review, CI/CD, incident response and Laura Tacho's argument that the real winners treat AI as an organisational and change-management problem rather than as a standalone coding tool.

1. Tools and harnesses: heterogeneity first, convergence later

While Anthropic's Claude Code is the early star and by far the most used Harness, OpenAI's Codex has seen a significant uplift over the past 6 months.¹⁶

The general curve seems as follows:

- **Early stage of adoption:** heterogeneity is often useful. Teams learn faster if engineers can try different tools and discover where each one actually helps.

¹⁶ Laura Tacho (DX), The Pragmatic Summit <https://youtu.be/LOHgRw43fFk?t=966>

- **Later stage / more mature operating model:** organisations tend to converge on a smaller tool set plus a few blessed workflows, because this makes security, support, training, and knowledge sharing more manageable.

The reviewed cases point in the same direction:

- **Mapbox** still allows tool choice, but in practice usage has converged strongly around Claude Code for IC workflows. At the same time, leadership pushes for fast tool approval and broad adoption.
- **Upvest** puts a strong emphasis on the governance: model choice is tied to data sensitivity, and the organisation uses an internal AI toolkit repo with Skills, Rules, Workflows, Agents, and profiles rather than leaving practice fully decentralized.
- **Atlassian** adds another important signal: internal agents are being extended beyond coding into review, CI/CD, incident handling, and intent-rich collaboration artifacts, which is much closer to an AI-native SDLC than to isolated assistant usage.
- **Cloudflare** points toward internal platforming rather than tool sprawl: the interesting asset is the governed stack around agents, not just the assistant itself.

So the pragmatic approach seems to be:

- **Do not force premature convergence while teams are still learning.**
- **Do not allow permanent tool sprawl either.**
- Let teams explore in a bounded way, then converge on the tool set and workflows that prove to be safest, most productive, and easiest to support.
- The harness itself becomes less important once the tooling around matures (internal platforming)

Spotify is a strong supporting example here: the talk argues that the scaling problem is orchestrating, verifying, standardizing and maintaining much larger volumes of generated change. Their answer is a broader platform stack around agents and developers: Backstage¹⁷, FleetShift¹⁸, trusted tools, CI/build verification, and strong technical standardization.¹⁹

2. Skills, agents, workflows

Once agentic usage spreads across an organisation, reusable building blocks become necessary: skills, rules, workflows, agent profiles, task-specific agents, review norms and verification patterns.

¹⁷ <https://github.com/backstage/backstage>

¹⁸ <https://engineering.atspotify.com/2023/04/spotify-shift-to-a-fleet-first-mindset-part-1>

¹⁹ Niklas Gustavsson (Spotify), Code with Claude <https://youtu.be/zFslvuvYifQ>

- **Upvest** describes an internal AI toolkit repo with Skills, Rules, Workflows, Agents, and profiles.²⁰
- **Mapbox** shows emerging patterns around structured specs, AI-assisted review, and role-specific tool usage.²¹
- **Atlassian** frames ownership as shifting away from code inspection alone toward intent artifacts, guardrails, and verification of outputs across the SDLC.
- **Spotify** adds a particularly practical example for large organisations: internal agent systems are paired with repo catalogues, migration orchestration, build verification, and standardization layers so that agents can do useful work safely across many teams and codebases.
- **Anthropic / Claude Code** adds the organisational-process angle: planning becomes more just-in-time, some rituals should be reduced rather than expanded, and teams should double down on shift-left verification, human review boundaries, and clear norms around ownership and decision-making.

The upvest approach of an AI toolkit seems to be a pragmatic approach for smaller engineering organisations without an existing developer platform organisation.

3. Knowledge sharing / Change management

Knowledge sharing is not a side topic. Since the adoption of agentic engineering is regarded as an operating model change, this rather has to be reframed as change management. Different approaches to managing this change - depending on company culture - can be observed: Top-Down or a combination of Top-Down and Bottom-up.

Independently of the approach taken, knowledge and context has to be shared among teams. This is visible at Mapbox through recurring AI knowledge-sharing sessions²² and at Upvest through the explicit toolkit / rules / workflows model.

There have been reports on token-rankings at big companies like Uber, Amazon or Meta to encourage engineers to adopt agentic engineering. Most companies - if not all - have stopped this approach and cancelled their AI leaderboards²³.

In the end general change management tenets apply: people need to understand the why, need to have the relevant support and resources and their concerns have to be taken seriously. Change takes time and speed of change needs to be balanced carefully with other metrics to prevent breaking points.

²⁰ Annex 2: [Upvest Case Summary](#)

²¹ Annex 3: [Mapbox Case Summary](#)

²² Annex 3: [Mapbox Case Summary](#)

²³ <https://www.businessinsider.com/amazon-ai-leaderboard-tokenmaxxing-2026-5>

Security and compliance

Specifically in this area size matters a lot. Bigger companies have platform engineering teams focused on developer productivity and approach security - like any other topic - from this lens. Smaller companies are rather following more pragmatic approaches.

Observed patterns include:

- sandboxes or isolated execution environments^{24 25}
- permissioned gateways for tools and data access
- review and approval policies that remain in force even with agent-generated changes
- logging and auditability for sensitive actions
- explicit data classification before allowing agent access

Upvest adds the complementary pattern of sensitivity-based model selection and controlled access via gateway-style integrations.²⁶

General tenets

One rule shows up repeatedly and should remain non-negotiable:

Every engineer is responsible for what they build with the agent. The agent is a new tool, not a new accountability boundary. Mapbox states this very explicitly: the code you commit is your code, even if Claude generated it.²⁷

The key security principle is simple: **the agent should not have broader access than the human using it.** Agent access should be mediated through identity, permissions, audit logs, and data classification. No secrets should live in the codebase, and no secrets should be exposed to the agent unless there is a controlled, auditable, and necessary reason.

The main attack classes are prompt injection, indirect prompt injection through documents or tickets, supply-chain attacks, over-broad tool permissions, context poisoning, insecure generated code, and data exfiltration via connected tools. These risks argue for bounded, inspectable workflows rather than free-form autonomy.

Summary

The strongest conclusion is: **Technical excellence matters even more in an agentic world.**

²⁴ Annex 4: [Uber](#)

²⁵ <https://blog.cloudflare.com/internal-ai-engineering-stack/>

²⁶ Annex 2: [Upvest](#)

²⁷ Annex 3: [Mapbox](#)

The teams that appear to benefit most from AI are usually the ones that already have solid CI/CD, strong automated tests, good observability, clear ownership, stable review practices, serious security posture, compliance and reliability embedded into infrastructure and enough standardization that both humans and agents can navigate the system effectively.

Agentic engineering increases the importance of these foundations.

Outlook

Two topics are currently being addressed rather marginally in the general discussion: 1. Token cost development and 2. Geopolitical constraints.

While the geopolitical constraints of coding agents are less relevant for American tech companies, the costs are already becoming a relevant topic, Uber only being the most outspoken about this topic.

It is hard to predict where token costs are ultimately heading since there are many competing constraints. LLMs do not have network effects and individual models have very little distinctive capabilities, hence ultimately the pricing power should not remain with the model provider but rather move upstream.²⁸ The model providers have incentives to move upstream and provide the app layer around the LLMs (Claude Code/Cowork, Codex etc.). This and the potential consolidation around this app layer can ultimately lead to strong pricing power and higher prices. Short term the increasing demand is forcing Anthropic to raise prices²⁹. It is also possible that the providers will reduce the limits included in the heavily subsidized subscription plans.³⁰

Using an open source harness can mitigate this risk but could come with certain limitations. The model providers are increasingly building tooling around their coding harnesses that includes cloud infrastructure and provides an environment for development that utilizes the security capabilities of the model providers and moves some of the security risks to the model provider's infrastructure. They also invest in the interfaces - instead of relying on the developer PC - agent tasks/development iterations can be managed via mobile apps.

Nevertheless, the commercial and geopolitical constraints could make running open source models in own infrastructure a viable solution for a lot of use cases.

1. Sovereign AI for the broad base

Self-hosted models such as Qwen 3.6 (27b or 35b), Gemma 4 (27b) or similar open-weight models are highly capable and can be used for the majority of low- to medium-risk internal use cases.

²⁸ Lenny's Podcast with Ben Evans <https://www.lennysnewsletter.com/p/a-rational-conversation-on-where>

²⁹ <https://letsdatascience.com/news/anthropic-revises-claude-enterprise-pricing-structure-f3022a32>

³⁰ <https://www.desightstudio.com/en/insights/claude-costs-who-really-pays-the-ai-bill>

Benefits are lower marginal cost, better control over data handling, easier alignment with privacy and sovereignty concerns. These models are a good fit for internal documentation, drafts and lower-risk engineering tasks

In a recent test of qwen-3.6-35b-A3B with 4 A100 GPUs we could not saturate the cluster with 16 concurrent users in llperf while generating 755 tokens per second (tps). Similar GPUs can be rented at an hourly rate of ~2€.

2. Metered use of frontier models for the hardest problems

Premium external model usage could be used for the hardest coding tasks, architectural reasoning, deep review assistance and critical summarization or synthesis tasks. This can create a more economically and strategically coherent model than going all in frontier APIs across everything. However, this cannot (yet) be observed in the industry.

References

- Laura Tacho (DX), The Pragmatic Summit: <https://www.youtube.com/watch?v=LOHgRw43fFk>
- Niklas Gustavsson (Chief Architect, VP Engineering Spotify), Code with Claude: <https://youtu.be/zFslvuvYifQ>
- Ty Smith & Anshu Chada (Uber), The Pragmatic Summit: <https://youtu.be/i1tZN41VKcE>
- HMZE - Beyond Vibe Coding podcast episode with Uli Strötz (Mapbox)
- HMZE - Beyond Vibe Coding podcast episode with Florian Thiel (Upvest)
- <https://blog.cloudflare.com/internal-ai-engineering-stack/>
- Thoughtworks: [The Future of software development retreat - key takeaways](#)
- Fiona Fong (Anthropic), Code with Claude <https://www.youtube.com/watch?v=IA5LWIGqnyM>
- Thomas Dohmke (Entire, prev. CEO Github), Rajeev Rajan (CTO, Atlassian), The Pragmatic Summit: <https://youtu.be/fYh1CWadxDM?t=110>

Supporting sources

1. <https://engineering.fb.com/2026/04/16/developer-tools/capacity-efficiency-at-meta-how-unified-ai-agents-optimize-performance-at-hyperscale/>
2. <https://openai.com/index/running-codex-safely/>
3. <https://www.businessinsider.com/amazon-ai-leaderboard-tokenmaxxing-2026-5>
4. <https://fortune.com/2026/05/26/uber-coo-ai-spending-tokens-claude-code/>
5. <https://finance.yahoo.com/sectors/technology/articles/ubers-coo-says-getting-harder-050841491.html>
6. <https://www.lennysnewsletter.com/p/a-rational-conversation-on-where>
7. <https://letsdatascience.com/news/anthropic-revises-claude-enterprise-pricing-structure-f3022a32>
8. <https://www.desightstudio.com/en/insights/claude-costs-who-really-pays-the-ai-bill>

Annex

1. Agentic Engineering Case Studies — Comparative Summary Report

This report summarizes public findings across six company case studies: Mapbox (Feb 2026)³¹, Upvest (May 2026)³², Uber (Feb 2026), Spotify (May 2026), Cloudflare (April 2026), and Anthropic (May 2026).

Executive Summary

Across the cases, agentic engineering is not primarily a story about faster autocomplete. The common pattern is organizational: once AI increases the amount of code, analysis, migration work, documentation, or maintenance output that can be produced, the bottleneck moves to verification, review, context, governance, cost control, and operating-model design. The companies unanimously report or imply increased throughput, but none of the mature examples treats output volume as sufficient proof of success.

The clearest shared lesson is that agentic engineering only scales when it is embedded into the engineering system. The companies all have solutions for access to repositories, CI, internal documentation, tickets, developer portals, registries, access controls, audit trails, and review workflows. Where public evidence is more detailed, this takes the form of gateways, MCP infrastructure, Backstage-like catalogs, internal CLIs, agent registries, generated repository instructions, sandboxed execution, and regulated SDLC gates. Where public evidence is less platform-specific, the same problem is addressed through review discipline, written culture, ownership norms, operational rigor, and explicit human judgment boundaries.

The main differences are therefore not about whether AI increases engineering capacity, but about which constraint each organization foregrounds. Some cases are primarily platform/control-plane stories, emphasizing centralized routing, identity, observability, sandboxing, and cost governance. Others are toil-automation stories, showing how background agents can execute migrations, upgrades, maintenance, and cleanup work at scale. A third group is more governance- or operating-model oriented, focusing on review, compliance, accountability, planning, and the changing role of engineers once code generation becomes cheap. Uber is most explicit in a critical perspective on ROI: it reports strong internal productivity signals, but also says that activity metrics are not yet sufficient business outcomes and that CFO-level justification becomes necessary as AI costs rise.

³¹ Annex 3: [Mapbox Case Summary](#)

³² Annex 2: [Upvest Case Summary](#)

Success Measurements

All detailed cases report or imply output improvements

The companies consistently report increased throughput, productivity, or automation capacity. This appears in different forms: higher PR or merge-request activity, more frequent AI-tool usage, stronger developer productivity sentiment, automated maintenance work at very large scale, faster internal tooling, and increased code or workflow output. Spotify and Cloudflare provide the strongest quantitative evidence, with Spotify reporting very high weekly AI-tool usage, strong productivity sentiment, substantially increased PR frequency, and millions of automated maintenance PRs through FleetShift, while Cloudflare reports broad R&D adoption, millions of AI requests, thousands of active internal users, and a material rise in weekly merge-request volume. Uber also reports clear throughput gains, including an earlier 10–15% developer-velocity bump from IDE-based AI assistance and a major shift toward Minion-powered agentic workflows. Mapbox and Upvest provide less formal benchmarking, but their public evidence still points toward higher throughput, faster tooling work, and increased PR or agent-assisted engineering activity.

Most companies agree that output volume is not the final metric

The shared caveat is just as important as the productivity signal: more generated code, more PRs, or more automated work does not automatically mean better engineering outcomes. The more mature cases converge on the view that review, verification, ownership, quality, and business impact become more important as AI increases output. Anthropic states this most generally by framing agentic coding as an operating-model challenge: planning, ownership, review, and verification norms have to change when code becomes easier to generate. Mapbox, Upvest, Uber, Spotify, and Anthropic all point in similar directions, even when they use different language.

Uber makes the distinction most sharply. It reports strong developer-experience and productivity signals, but explicitly argues that activity metrics are not business outcomes. From this perspective, generated diffs, usage rates, and developer satisfaction are useful intermediate measures, but the real question is whether AI accelerates product delivery, experimentation, revenue-relevant work, or measurable business impact. This is why Uber is instrumenting the pipeline from design creation to experiment launch rather than relying only on code-volume metrics.

Only Uber publicly highlights unresolved ROI at this level of specificity

Several cases imply that outcome measurement is difficult, but Uber is the only one that publicly frames the ROI problem with this level of directness. Its evidence is not negative; it is more mature and more uncomfortable. Uber reports strong adoption and productivity signals while

also saying that AI costs have risen sharply since 2024 and that cost justification has moved beyond normal engineering budgets. The case therefore highlights the next measurement frontier for agentic engineering: connecting usage, generated output, developer sentiment, and velocity to durable business outcomes.

Adoption Policy

Successful adoption is not just license rollout

Across the cases, successful adoption depends less on giving engineers access to tools and more on embedding AI into the existing engineering system. Broad usage emerges when agents can interact with repositories, tickets, CI, internal documentation, developer portals, standards, and approved tools in a way that makes delegation useful and review tractable. The more platform-oriented companies achieve this through shared gateways, registries, CLIs, Backstage-like systems, MCP infrastructure, and standardized workflows. The more governance-oriented cases embed AI into existing SDLC, review, test, data-handling, and compliance rails. In both versions, adoption is a systems-design problem, not a procurement problem.

Leadership support helps, but peer proof matters

Leadership support clearly helps normalize AI usage, especially when companies move from permissive experimentation to expected or strategically framed adoption. However, the evidence also suggests that top-down pressure is not enough. Agentic engineering requires engineers to change how they work: from direct authorship toward delegation, planning, supervision, and review. Uber's public account is especially useful here because it distinguishes between directives that increase adoption metrics and peer examples that actually change behavior. Engineers appear more likely to adopt agentic workflows when they see credible colleagues use them successfully on real work.

Companies differ in how prescriptive they are

The companies differ less in whether they support AI adoption than in what they prescribe. Some are culturally prescriptive, treating AI openness, written practices, review habits, and operational discipline as part of the engineering culture. Others are platform-prescriptive, requiring usage to flow through approved routing, authentication, gateway, registry, and MCP infrastructure. Regulated environments are compliance-prescriptive: teams may retain autonomy, but review, deployment, data classification, and access-control requirements remain binding. The least mandate-driven examples still achieve adoption by making the platform useful enough that engineers naturally pull it into their work. Anthropic is distinct because the public evidence focuses less on rollout mechanics and more on how the operating model changes inside a team already working with Claude Code as a default practice.

Harness Governance

The mature cases put agents inside governed harnesses

The mature pattern is consistent: agents need a governed harness. They require access to code, documentation, issues, build systems, internal tools, and organizational context, but that access must be mediated. In the most detailed cases, the harness includes model gateways, MCP gateways, internal CLIs, registries, CI execution, sandboxing, centralized configuration, telemetry, logging, cost controls, identity, access management, and review artifacts. The purpose is not only security; it is also repeatability, observability, maintainability, and confidence in the generated work.

The differences reflect organizational context. Platform-heavy cases emphasize centralized control planes for routing, authorization, observability, and tool access. Engineering-orchestration cases focus more on background agents, build verification, developer-platform integration, and migration workflows. Regulated cases treat the harness as an extension of SDLC governance, adding stronger requirements around SSO, role-based access, auditability, data classification, human PR review, and controlled production or log access. These are different implementations of the same underlying requirement: agentic work has to be operationalized, not merely prompted.

Mapbox and Anthropic focus more on norms than public platform architecture

Mapbox and Anthropic provide less public detail on centralized platform architecture than the more infrastructure-heavy cases, but that should not be interpreted as absence of governance. The available evidence describes their governance more through culture, review boundaries, operational discipline, and human judgment. Mapbox's case emphasizes written culture, review practices, OPEX discipline, post-mortems, and customer-impact thinking. Anthropic's case emphasizes where human judgment remains essential: security, legal, trust, product judgment, domain expertise, planning, and ownership. The contrast is therefore about public evidence and emphasis: some companies expose the control plane; others expose the operating norms.

Tool-Sharing

Shared context becomes infrastructure

A strong shared pattern is that tool-sharing and context-sharing become infrastructure once agentic usage spreads. Local prompt craft is not enough when many engineers depend on agents for real work. Companies need reusable workflows, shared registries, standard configurations, catalogs, repository instructions, migration templates, static-analysis rules,

knowledge systems, and feedback loops. In the more systematized cases, this appears through MCP registries, internal CLIs, Backstage integration, generated repository-level agent guidance, AI toolkits in version control, standardized agent surfaces, CI-connected automation, and links between generated work, logs, issues, test plans, and review artifacts.

The strategic point is that context becomes a first-class engineering asset. Agents become more useful when they can access company-specific conventions, architecture, ownership, tickets, service metadata, standards, and historical decisions. But making that context available creates new responsibilities: it has to be maintained, permissioned, versioned, and made legible to both humans and machines.

Mapbox and Anthropic rely more on practice sharing and operating norms

The more culture-oriented cases share knowledge through recurring sessions, written practices, prompt and review conventions, spec-driven development groups, prototype-based decision-making, and explicit norms around ownership and verification. This is a meaningful difference from the platform-heavy cases, which institutionalize tool-sharing through systems. Both approaches solve part of the same problem. System-based sharing scales access and consistency; practice-based sharing helps engineers develop judgment about when to delegate, when to inspect, and where human responsibility cannot be outsourced.

Security and Compliance

All mature cases preserve human accountability

The common security and compliance principle is clear: AI does not remove accountability. It increases the need for access control, review, verification, auditability, and explicit human ownership. Every mature case preserves human responsibility in some form, whether through PR review, CI, tests, incident processes, post-mortems, customer-impact analysis, legal and trust boundaries, platform guardrails, or regulated SDLC controls. The mechanisms differ, but the direction is the same: as output becomes cheaper, accountability has to become more explicit.

Upvest and Cloudflare have the strongest explicit compliance/security models

The most explicit security and compliance models combine policy with technical enforcement. In the regulated case, controls are shaped by auditable human review, restricted production and log access, controlled break-glass procedures, SSO, role-based access, data and document classification, impersonation models, and strict boundaries around sensitive data. In the infrastructure-platform case, controls are shaped by Zero Trust access, AI Gateway routing, provider-key management, model-catalog control, non-retention settings, MCP identity, sandboxed execution, and automated AI code review in CI. The shared lesson is that

governance becomes more scalable when it is implemented as infrastructure rather than left as informal guidance.

Uber has substantial platform controls but less public compliance detail

Uber is clearly a governance and platform-control case, but the public evidence is stronger on operational controls than on formal compliance detail. It describes centralized MCP authorization, telemetry, logging, registries, sandboxing, internal CI execution, controlled repository and tool access, and traceable artifacts connected to Slack, PRs, logs, Jira tickets, test plans, and agent identity. However, the available public material does not give the same level of detail on secrets handling, production-access policy, data retention, or formal compliance review. Uber also explicitly warns that review load becomes a risk as generated-code volume rises: if planning and code review do not scale, bugs can slip through despite higher output.

Other Noteworthy Patterns

Technical excellence compounds AI leverage

A robust pattern across the cases is that AI works best where the engineering system is already mature. Agentic workflows benefit from strong CI, clear ownership, internal platforms, service catalogs, operational discipline, migration tooling, review norms, standards, documentation, and incident practices. The uncomfortable implication is that agentic AI does not magically compensate for weak engineering foundations. It amplifies the surrounding system. Strong platforms and disciplined engineering cultures turn agents into leverage; weak foundations turn them into additional entropy.

Review and verification become the bottleneck

The companies converge strongly on review and verification as the next bottleneck. Once code generation becomes cheap, the scarce capacity shifts toward deciding what should be built, checking whether it is correct, understanding system context, reviewing changes, validating behavior, and owning consequences. Mapbox states the review bottleneck directly. Upvest emphasizes human-in-the-loop review pressure. Spotify reframes the constraint as orchestration and verification rather than code writing. Uber reports that developers spend more time planning and reviewing as generated-code volume increases. Anthropic generalizes the issue into a redesign of review, ownership, and verification norms. The durable finding is simple: code generation is not the hard part for long; safe integration is.

Bounded toil work is the clearest near-term win

The best evidenced near-term wins are bounded, verifiable, and operationally integrated. Migrations, upgrades, bug fixes, documentation updates, test generation, dead-code cleanup, internal tooling, and maintenance automation are easier to delegate because they have clearer

start and end states than ambiguous product work. Uber and Spotify provide the strongest evidence here through agentic toil workflows, background agents, maintenance PRs, and migration automation. This does not mean agents cannot support product development, but the public evidence is strongest where the work is constrained, repeatable, and reviewable.

Context is the hidden infrastructure problem

Every serious case eventually runs into context. Agents need company-specific rules, domain knowledge, service ownership, architecture, tickets, documentation, Slack or collaboration context, datasets, internal tools, and deployment conventions. The implementation differs: some companies use Backstage-like systems, MCP registries, generated AGENTS.md files, internal AI toolkits, written culture, or architecture documentation. The underlying lesson is shared: without organizational context, agents remain generic; with context, they become useful. But context access introduces governance, security, freshness, and maintenance problems that have to be treated as engineering work.

Consolidated Lessons

The consolidated lesson is that output improvements are real but insufficient. The companies consistently report higher throughput, stronger productivity sentiment, or more automation, yet the harder problem is proving quality, business impact, and sustainable delivery. Uber is the clearest ROI warning because it combines strong internal productivity signals with the explicit claim that activity metrics are not business outcomes and that rising AI costs require stronger justification.

Agents need platforms, not just prompts. Serious adoption depends on gateways, registries, CI, catalogs, MCP infrastructure, standardized configuration, controlled tool access, shared context, and traceable artifacts. Governance is not a blocker when it is implemented technically; it becomes an accelerator by making broader use safer and more repeatable. Human accountability remains non-negotiable, but it has to be supported by redesigned review, verification, testing, ownership, and planning practices.

The most evidenced starting point is bounded toil: migrations, upgrades, maintenance, documentation, test generation, cleanup, bug fixing, and internal tooling. These workflows are easier to scope and verify than open-ended autonomous feature development. At the same time, shared context becomes a first-class asset: Backstage, AGENTS.md, MCP registries, AI toolkits, written culture, and architecture documentation are different implementations of the same need. Finally, cost control has to be designed early. Agentic workflows can consume far more tokens and infrastructure than classic autocomplete or chat, so routing, model selection, measurement, caching, and optimization become part of the operating model.

Evidence Caveats

The evidence depth differs by company. Cloudflare provides the most detailed first-party infrastructure and usage data. Uber's revised case is grounded in a primary conference talk and is operationally detailed, especially on platform controls, agentic workflows, cost, and ROI pressure. Spotify's figures are also taken from a talk of its Chief Architect and VP Engineering. Mapbox and Upvest are rich qualitative cases based on podcast interviews. Anthropic provides strong operating-model insight but little quantitative benchmarking.

The report therefore compares verified public information, not actual company maturity in absolute terms. Absence of evidence in a public talk, article, or interview should not be read as evidence that a company lacks a practice internally.

2. Upvest — Agentic Engineering Case Summary

Source: [HMZE - Beyond Vibe Coding podcast episode with Florian Thiel \(Upvest\)](#), 2026-05-22.

Success Measurements

Upvest combines qualitative engineering judgment with increasingly formal Developer Experience and productivity measurement.

- **DX surveys:** Upvest uses DX to measure developer satisfaction, interruptions, meeting load, platform quality, local development quality, and AI tooling support.
- **AI usage signals:** DX sensors can inspect agent interactions, AI-generated code flow, intervention/rejection patterns, and correlations with throughput and DORA-like metrics.
- **Throughput:** Florian's qualitative read is that PR frequency and produced PRs have increased, although the company has not yet completed a formal retrospective analysis over historical GitHub data.
- **Quality:** There are reports of localized quality degradation, especially in domains where the agent has poor context or where model/tool quality is weaker.
- **Sentiment:** AI tooling satisfaction is very high, reportedly in the 90% range in DX surveys, especially after recent model improvements.
- **Outcome measurement remains hard:** Upvest can observe more code and more throughput, but Florian is explicit that proving better business outcomes remains the "holy grail."

Selected podcast quotes, translated:

"The PR frequency and the produced PRs have increased - that is my gut feeling and what you hear in conversations with engineers." - Florian Thiel, ~53:22

“The holy grail - whether we can actually observe better outcomes - is of course not solved 100% by this either.” - Florian Thiel, ~48:35

“Overall, especially in the last two or three months since Opus, sentiment has gone up massively. What we also see in the DX surveys is that satisfaction with AI tooling as support is very high, in the 90-something percent range.” - Florian Thiel, ~54:42

Adoption Policy

Upvest’s adoption model is pragmatic and governance-aware rather than pure top-down standardization.

- AI adoption builds on an already code-first, approval-driven engineering organization.
- Team autonomy remains important; teams can choose locally appropriate tools and workflows.
- At the same time, central SDLC gates are technically enforced: human approval, test stages, deployment rules, and GitHub Actions.
- Regulated-company constraints shape adoption: Gemini is used more for internal documents, while Claude Code is preferred for coding.
- Agentic implementation can be relatively autonomous when output still flows through human review and approval gates.

Selected podcast quotes, translated:

“We have the big advantage that we started relatively code-first and approval-driven. All our pull requests have human review; that is technically enforced and auditable.” — Florian Thiel, ~09:38

“In the end, there is always a human who stands behind the change.” — Florian Thiel, ~09:38

“If there is always an approval gate at the end, then you can be much more relaxed about giving agents a bit more freedom.” — Florian Thiel, ~18:26

Harness Governance

Upvest’s harness governance is explicitly tied to data sensitivity, compliance, and practical tool strength.

- **Claude Code** is the preferred and strongest tool for software development.
- **Claude Chat** is also used, but access to internal documents and customer-sensitive contexts is more constrained.

- **Gemini** is used more for internal documents because of Zero Data Retention and European data-center requirements.
- **Gemini CLI** exists in the environment but is not considered as strong as Claude Code today.
- **OpenSpec** is used by Florian personally for spec-driven work.
- **MCP Gateway**: Upvest uses a Cloudflare-based MCP Gateway with custom additions for impersonation, SSO, role-based access, and SaaS integrations such as Datadog and Linear.
- **OpenCode** is mentioned as one of the harnesses into which the AI Toolkit can be hooked.
- **GitHub/GitHub Actions** enforce key SDLC constraints such as independent review.

Selected podcast quotes, translated:

“Today it is Claude Code, Claude Chat, and a bit of Gemini. We have a somewhat fragmented environment because, as a regulated company with a lot of customer data, we have compliance requirements that are not trivial.” — Florian Thiel, ~08:16

“Everything that touches internal documents is more in the Gemini world. Everything that is code is Claude Code.” — Florian Thiel, ~08:16

“What we need is impersonation for all MCPs. [...] Access always happens in the name of the person operating the agent.” — Florian Thiel, ~14:36

Tool-Sharing

Upvest has one of the clearest tool-sharing patterns among the reviewed cases.

- The core mechanism is an internal **AI Toolkit** in a Git repository.
- The toolkit contains Skills, Rules, Workflows, Agents, and reusable profiles.
- It started bottom-up from ambitious engineers rather than as a heavy top-down platform project.
- It contains general software engineering knowledge plus Upvest-specific rules, such as signed commits.
- Profiles package skills, agents, and rules for domains such as Data Engineering or Go Engineering.
- Engineers can load these profiles into their Claude session or customize personal profiles.
- The hardest remaining problem is encoding company-specific context: architecture patterns, business context, domain knowledge, and the right dose of information for agents.

Selected podcast quotes, translated:

“The AI Toolkit is basically a Git repo with Skills, Rules and Workflows.” — Florian Thiel, ~26:22

“A few ambitious people in the company started it and distilled a lot of general software knowledge.” — Florian Thiel, ~26:22

“It has some Upvest-specific skills. It also has rules that we always use signed commits — things that are not obvious elsewhere, but specific for us.” — Florian Thiel, ~26:22

“The most challenging part is getting the specific company context in there: what are our fundamental architecture patterns, what is important for the business?” — Florian Thiel, ~27:50

Security and Compliance

Upvest treats security and compliance as technical rails for agentic engineering rather than as after-the-fact policy guidance.

- Regulated FinTech requirements are not only a constraint; they become a foundation for safer AI usage.
- Human review and pull-request approvals are technically enforced and auditable.
- Access to production/log data is controlled and restricted; direct log access is not generally available.
- Datadog MCP access is configured so personally identifiable data is not generally accessible.
- Incident-time “break glass” procedures exist, but are controlled and logged.
- MCP access uses impersonation: agents receive the same access limits as the human using them.
- SSO sessions expire regularly and must be refreshed.
- Data streams and documents are increasingly classified by sensitivity, enabling role-based data access and preventing agents from seeing information they should not see.
- Google Workspace document classifications range from secret to internal/company use to external.

Selected podcast quotes, translated:

“From a compliance perspective, the difference between ‘the agent wrote the code’ and ‘the human wrote the code’ is not really there. In the end, a human stands behind the change.” — Florian Thiel, ~09:38

“There is no direct access to log files for anything. Access, especially to live systems, is very controlled and restricted.” — Florian Thiel, ~13:22

“If I have access to personal data because of my role, then I can see it. [...] If not, the agent cannot magically see it either.” — Florian Thiel, ~14:36

“If data classification exists from the beginning, you avoid quickly moving into a range that is not appropriate in a regulated environment.” — Florian Thiel, ~14:36

Other Noteworthy Information

- **Regulation as advantage:** Upvest’s existing compliance, approval, audit, SSO, and data-classification foundations make agentic work safer rather than merely harder.
- **Small tech stack:** Upvest intentionally keeps its stack narrow: Google Cloud/Workspace, Go, PostgreSQL, Kafka, adapter services, and clear external contracts.
- **Domain complexity:** Investment infrastructure is hard to agentify because end-to-end flows cross many services, banks, trading venues, reconciliation processes, and compensating bookings.
- **Test harnesses are the next frontier:** Better end-to-end test signals and reproducible environments are required before agents can run autonomously for longer stretches.
- **Containerized agents are desired but not fully there:** Upvest is working toward quickly reproducible, protected environments; load-test environments are a stepping stone.
- **Documentation is becoming more valuable:** Architecture documentation, diagrams, and the “why” behind decisions are increasingly important because agents need the same context humans need.
- **Agent sprawl is a major pain:** Linear, Slack, Gemini, Claude, and others each bring their own agents, skills, and context. Reuse and platformization are still weak.
- **Context management is strategic:** Florian expects reusable context management and test harnesses to become important near-term product categories.

3. Mapbox - Agentic Engineering Case Summary

Source: [HMZE - Beyond Vibe Coding podcast episode with Uli Strötz \(Mapbox\)](#), 2026-02-27

Success Measurements

Mapbox appears to measure AI success less through a single formal AI KPI and more through observable operational and delivery signals:

- **Delivery speed:** Uli describes a clear compression of delivery timelines: projects that would previously have taken years now happen in months.
- **Internal tooling output:** The strongest visible productivity gain is in internal tooling. Tools that would previously have required annual planning or a dedicated team are now often built immediately, sometimes as disposable validation tools.
- **Review pressure:** The main negative success signal is that review has become the bottleneck. AI increases code volume, but human review capacity has not scaled at the same rate.
- **Operational health:** Mapbox's existing OPEX culture provides strong qualitative guardrails: incidents, customer impact, post-mortems, RCAs, action items, SLAs, and weekly operational reviews remain central.
- **Customer impact:** Operational discussions start from customer impact, not from internal activity metrics.

Selected podcast quotes, translated:

"Almost all the code we produce is now co-authored by Claude. I rarely see a commit that still comes from humans alone." — Uli Strötz, ~29:15

"We are producing an enormous amount of code. That is great, that is what we wanted. But we still have to review it. And that is currently the biggest blocker, the productivity blocker." — Uli Strötz, ~29:15

"I do not have clear numbers, but we deliver projects in a timeframe that used to take years, now in months. You see it most clearly in tooling." — Uli Strötz, ~36:05

"The tools just get built. [...] Often these are throwaway websites. They are built once, looked at briefly, and either they serve the need or they are good enough that we discuss how to integrate them into the official toolchain." — Uli Strötz, ~37:36

Adoption Policy

Mapbox is pushing AI adoption aggressively from leadership downwards.

- AI started as "approved to use," then shifted into a clear expectation to use AI as much as possible.
- Tool approval is expected to be fast; if a tool is not approved within roughly 24 hours, leadership escalation is possible.
- AI openness is now part of the hiring process: candidates go through an explicit AI interview step to test openness, maturity, and practical AI experience.
- The cultural message is blunt: people who categorically refuse AI in production code no longer fit the current Mapbox expectation.

Selected podcast quotes, translated:

“In the last six months there has been a strong shift where the message is very clear: use everything that works.” — Uli Strötz, ~17:23

“If a tool is not approved within 24 hours, there needs to be a very clear reason. You can go to leadership and escalate if you do not get access to a tool.” — Uli Strötz, ~17:23

“The expectation is that everyone uses AI maximally.” — Uli Strötz, ~17:49

Harness Governance

Mapbox does not appear to enforce a single rigid harness, but practice is converging around Anthropic tools.

- **Claude Code** has become the de facto default for many individual contributor workflows.
- **Claude Desktop** is Uli’s primary management interface across Slack, Google Docs, Confluence, Jira, meeting transcripts, performance reviews, interview feedback, and context briefings.
- **Gemini** is used primarily for meeting transcripts and parts of the Google ecosystem.
- **Rovo AI** is used experimentally in Jira; Uli found it strong for restructuring tickets but weak for higher-level reporting.
- The governance model is therefore not strict standardization, but leadership-backed openness with practical convergence around the tools that work best.

Selected podcast quotes, translated:

“For developers at Mapbox, Claude Code is the most important tool. They cannot work without it anymore. For me, Claude Desktop is my most important partner, my most important employee.” — Uli Strötz, ~12:09

“It is not a corset. You can choose freely. But I notice that almost everyone uses Claude Code. In my closer environment, it has clearly established itself over the last few months.” — Uli Strötz, ~22:00

Tool-Sharing

Mapbox’s tool-sharing pattern is currently more emergent than platformized as of the time of recording.

- There are weekly AI knowledge-sharing sessions.

- Developers share practices around Claude Code, prompt patterns, PR visualization, review support, and spec-driven development.
- A “spec-driven development” group experiments with YAML or Markdown specs that are committed and used as input for Claude.
- Internal tools are often created bottom-up. If a disposable tool proves useful, it may move into a more official productization or toolchain-integration path.
- Written culture is strong: documents, design docs, post-mortems, RCAs, and structured textual artifacts are central. AI is used to draft, refine, visualize, and compress these artifacts.

Selected podcast quotes, translated:

“We have weekly knowledge-sharing sessions about AI development, and you can see everything moving toward Claude Code.” — Uli Strötz, ~22:00

“There is a group inside Mapbox called spec-driven development. [...] You use AI to make the specs very clear first.” — Uli Strötz, ~34:01

“At Mapbox everything has to be written down as text. No bullet points. Diagrams can only support; the written document is the source.” — Uli Strötz, ~47:51

Security and Compliance

Mapbox’s most explicit control mechanism is accountability plus operational discipline.

- The engineer remains accountable for committed code, even if Claude generated it.
- Human review remains mandatory in the current workflow, although it is under pressure.
- Mapbox has not yet seen an incident caused by insufficiently reviewed AI-generated code, but Uli expects this risk to materialize eventually.
- The company relies heavily on existing tests, review, OPEX, incident management, post-mortems, RCA discipline, customer-impact analysis, and 30-day SLAs for post-mortem action items.
- For high-risk production systems, Mapbox’s strong operational culture acts as a compensating control against uncontrolled AI acceleration.

Selected podcast quotes, translated:

“The code you commit is your code. Even if it comes from Claude, you are responsible. You must be able to answer questions about it.” — Uli Strötz, ~29:15

“We do not accept the answer: Claude produced it, it is not my code.” — Uli Strötz, ~29:15

“There has not been a single incident because code generated by AI was not reviewed. But I am waiting for it to happen. I think it is only a matter of time.” — Uli Strötz, ~39:58

Other Noteworthy Information

- **Review is the new bottleneck:** Mapbox is exploring Claude-assisted review, HTML PR visualizations, architecture visualizations, smaller PRs, better commit structure, and possibly a future shift away from classic PR review.
- **PR visualization:** Large PRs are converted into HTML pages that explain architecture changes and provide context for reviewers.
- **OPEX culture is a major asset:** Incidents are categorized, reviewed in front of senior leadership, and translated into concrete action items.
- **Post-mortem discipline:** Action items from post-mortems become Jira tickets with a 30-day SLA; unresolved items make the team visibly red on scorecards.
- **RCA depth:** Bug tickets are not simply fixed; teams first clarify customer expectation, actual behavior, whether it is truly a problem, and why the system behaves that way.
- **AI as management leverage:** Claude Desktop is used for performance reviews, meeting prep, interview feedback, management self-improvement, document drafting, and context synthesis.
- **Prediction:** Uli suspects classic PR review may change substantially; in his words, the current review process is increasingly in the way.

4. Uber — Agentic Engineering Case Summary

Source: *Recording of Talk at The Pragmatic Summit, 2026-03-10:*

<https://www.youtube.com/watch?v=i1tZN41VKcE>

Success Measurements

Uber is no longer merely an ROI-warning case. The talk gives concrete evidence that Uber sees strong internal productivity impact from agentic engineering, while still treating business-outcome attribution and cost as unresolved measurement problems.

- **Strategic framing:** Uber’s CEO Dara Khosrowshahi frames AI as one of Uber’s strategic shifts: moving from a human / early-AI-powered company toward a generative- or agentic-AI-powered company.
- **Human augmentation, not replacement:** The explicit goal is to augment human productivity and free engineers from toil so they can focus on creative product work.
- **Classic IDE AI uplift:** Uber reports that earlier pair-programming-style IDE assistance produced roughly a **10–15% bump in overall developer velocity**.

- **Agentic shift:** Uber sees the larger shift in asynchronous “peer programming,” where developers delegate work to background agents, redirect them, and review the output.
- **Toil concentration:** Once agentic workflows became available, **70% of workflows submitted to agents were toil tasks**, because tasks such as library upgrades, migrations, bug fixes, documentation, and dead-code cleanup have clearer start/end states and higher success rates than ambiguous product work.
- **Developer experience:** Uber reports that internal developer NPS, developer experience, self-reported satisfaction, self-reported productivity, AI-landed code, and overall engineering velocity are at very high levels.
- **Power-user divergence:** After Minion and stronger models such as Sonnet and Opus became available, the gap widened between casual users and power users who use the tools heavily.
- **Business-outcome attribution remains unsolved:** Uber explicitly says activity metrics are not the same as business outcomes. The CFO asks for impact on revenue, not diffs. Uber is instrumenting the feature pipeline from design creation to experiment launch to measure whether AI accelerates product delivery.
- **Cost pressure is real:** Uber says AI costs have increased at least **6x since 2024**, moving from something fundable inside an engineering budget to something requiring CFO-level justification. This drives model-selection and token-efficiency work.

Selected quotes from the talk transcript:

“What we're not pushing for is AI automate all humans in the company... we want to focus on enabling our engineers to focus on creative work rather than toil.”

“We saw about a maybe a 10 to 15% bump in overall dev velocity.”

“As soon as we made our agentic workflows available to developers, what we saw is 70% of the workflows that developers are pushing into the system were toil tasks.”

“These are activity metrics. Right? These are not necessarily business outcomes.”

“The cost of AI is too damn high... since 2024, our costs have gone up at least 6x.”

Adoption Policy

Uber combines strategic top-down support with a more bottom-up adoption mechanism based on peer examples and internal champions.

- **Executive support:** AI is positioned as one of Uber's six strategic shifts.
- **Broad enablement ambition:** Uber wants all work at Uber to be supportable by generative AI where it augments human productivity.

- **Best-of-class openness:** Uber wants engineers to experiment with the latest industry agents and clients, including tools such as Claude Code, Codex, and Cursor.
- **Adoption is behaviorally hard:** Uber notes that developers are used to authoring code directly in their IDEs; asking them to delegate work to agents requires a real workflow change and feels risky.
- **Top-down mandates had limited effect:** Leadership directives to adopt AI moved metrics, but were not the most effective mechanism.
- **Sharing wins works better:** Uber found that adoption accelerated when engineers shared concrete successful examples with peers. Engineers trust other engineers more than directors.
- **Demonstrability matters:** In one demo, four VPs landed code in 24 minutes after not having done so for years, showing how important visible demonstrations are for adoption.

Selected quote from the talk transcript:

“The more successful technique that we've applied is actually just sharing wins... key promoters pushing techniques to their peers, because those promoters are typically engineers, and engineers trust other engineers.”

Harness Governance

Uber has a substantial internal harness/governance stack. The public talk describes a layered platform rather than a single coding assistant.

- **Existing AI foundation:** Uber builds on its historical Michelangelo ML platform, which provides model gateway capabilities, internal model hosting, inference/training infrastructure, and access to frontier providers such as OpenAI and Anthropic.
- **Organizational context:** Effective agents need access to source code, engineering documentation, Jira tickets, Slack information, internal services, datasets, and other organizational memory.
- **MCP strategy:** Uber rapidly adopted MCP and created a central MCP Gateway through a cross-company tiger team.
- **MCP Gateway responsibilities:** The gateway proxies internal and external MCPs through Uber's service infrastructure and provides consistent authorization, telemetry, logging, registry, and sandboxing.
- **Agent registry and builders:** Uber supports building agents via SDKs and no-code solutions, with visualization, telemetry, tracing, reuse, and discovery through registries.
- **Multi-client support:** Uber supports multiple agent clients rather than a single harness, including Claude Code, Codex, Cursor, and internal specialized agents.
- **AIFX CLI:** Uber built AIFX CLI as the central developer-facing interface for provisioning/configuring agent clients, installing/discovering MCPs from registries,

applying standard configurations, improving default prompts, and connecting to background-task infrastructure.

- **Minion:** Minion is Uber's formal background-agent platform. It runs on Uber's internal CI infrastructure, has monorepos checked out, has network access to internal infrastructure, connects to MCP servers through AIFX, and exposes web, Slack, GitHub PR, CLI, and API interfaces.
- **Own-infra bootstrap:** Uber built Minion on its own infrastructure because external background-agent products ran in third-party infrastructure; running internally allowed faster bootstrapping with existing repos, network access, and CI.

Selected quote from the talk transcript:

"We ended up putting a tiger team together from across the company that came together and designed a strategy and built a central MCP gateway."

"Minion is our formal background agent platform... It runs on our CI platform. This has our mono repos checked out... handles all of the network access into the rest of the infra."

Tool-Sharing

Uber's tool-sharing is platform-oriented: shared registries, shared configuration, shared MCP infrastructure, and common interaction surfaces.

- **MCP registry:** Engineers can discover, install, configure, and test MCPs through a central registry and sandbox.
- **Agent registry:** Agents built by teams can be made reusable and discoverable for other engineers or non-engineering users.
- **Standard configuration management:** AIFX CLI distributes standard configurations so new users get more effective prompts and agent setups immediately.
- **Prompt improvement:** Minion includes a prompt improver that analyzes a prompt and suggests changes to increase the chance of successful task execution.
- **Shared interaction surfaces:** Developers can interact with Minion through web UI, Slack, GitHub PRs, CLI, and APIs.
- **Observable artifacts:** Minion creates links to PRs, artifacts, logs, Jira tickets, test plans, agent identity, and follow-up/retry flows.
- **Peer-sharing adoption loop:** Uber explicitly uses shared internal examples and engineer-to-engineer promotion to spread effective techniques.

Selected quote from the talk transcript:

"The ability to install and discover MCPs from the registry, configure those inside of the agent clients, deploy standard configuration management..."

Security and Compliance

The talk does not provide a full compliance model, but it does describe several technical governance mechanisms.

- **Centralized authorization:** The MCP Gateway handles authorization consistently for internal and external MCP access.
- **Telemetry and logging:** MCP access and agent workflows are instrumented with telemetry and logging.
- **Sandboxing:** Uber provides a sandbox for developers to test MCP behavior before using it in workflows.
- **Internal CI execution:** Minion runs on Uber's internal CI platform, which gives controlled access to repositories, infrastructure, network, and internal tooling.
- **Traceability:** Minion links tasks to Slack notifications, PRs, artifacts, Jira tickets, logs, test plans, and the agent used.
- **Review pressure remains a risk:** Uber notes that as more code is generated, developers spend more time in planning and code review. This can slow them down or let bugs through if review load becomes too high.
- **Incomplete:** The talk does not give detailed public information on secrets handling, production access policies, data-retention terms, formal compliance review, or exact approval gates.

Selected quote from the talk transcript:

"This allows us to both proxy external and internal MCPs... in a consistent way to engineers that handle things like authorization, telemetry, logging, everything else that you might expect."

Other Noteworthy Information

- **Uber is a strong platform case:** The new primary source shows Uber is much closer to Cloudflare, Spotify, and Upvest in platform maturity than the earlier Fortune-only summary suggested.
- **The best current use case is toil:** Uber explicitly says agentic workflows work best where start and end states are clear: upgrades, migrations, bug fixes, documentation, and dead-code cleanup.
- **Workflow changes from authorship to orchestration:** Developers increasingly act as tech leads for multiple agents rather than pure code authors.
- **Planning and review expand:** The workflow shifts time away from manual code authorship toward planning, prompt/context setup, code review, and validation.
- **Legacy/internal context is hard:** Uber notes that integrating 10–15 years of sophisticated and archaic internal systems into AI-reachable MCP endpoints is difficult.

- **Cost optimization becomes infrastructure work:** Uber wants infrastructure to pick the right model for each part of the job, e.g., stronger models for planning and lower-cost effective models for execution, without forcing developers to reason about it manually.
- **Earlier Fortune article now fits as a caveat, not the main story:** The article's concern about ROI and cost is consistent with the talk, but the talk provides much richer primary evidence about Uber's internal platform and measured productivity impact.